

IPC

```
manishaa@localhost:~$ # IPC (Inter-Process Communication) COMMENTS - PART 1

# IPC allows processes to communicate and synchronize with each other.

# 1. PIPE
# Unidirectional communication channel
# Used to send data from one process to another
# Created using pipe() system call or '|' shell operator

# Create a pipe
int fd[2];          # fd[0] -> read end, fd[1] -> write end
pipe(fd);           # Returns 0 on success

# Example using shell
ls -l | grep ".txt" # Output of ls is sent to grep through a pipe

# 2. NAMED PIPE (FIFO)
# Like pipe, but exists in filesystem
# Allows communication between unrelated processes
mkfifo myfifo       # Create a FIFO special file
# One process writes
echo "Hello" > myfifo # Writer
# Another process reads
cat myfifo           # Reader
manishaa@localhost:~$
```

```
manishaa@localhost:~$ # IPC (Inter-Process Communication) COMMENTS - PART 3

# 5. SHARED MEMORY
# Fastest IPC mechanism
# Multiple processes access the same memory segment

ipcmk -M 1024        # Create shared memory segment (1024 bytes)
ipcs -m              # Show shared memory segments
ipcrm -m <shmid>     # Remove shared memory segment

# Attach and detach (in C)
int shmid = shmget(key, size, 0666); # Get shared memory id
char *addr = shmat(shmid, NULL, 0);  # Attach to shared memory
shmdt(addr);                  # Detach from shared memory

# 6. SEMAPHORES
# Used for process synchronization
# Control access to shared resources
ipcmk -S 1           # Create semaphore
ipcs -s              # Show semaphores
ipcrm -s <semid>     # Remove semaphore

manishaa@localhost:~$
```

```
manishaa@localhost:~$ # IPC (Inter-Process Communication) COMMENTS - PART 2

# 3. SIGNALS
# Used to notify a process about an event
# Asynchronous communication
# kill(), sigaction(), signal() are used

kill -9 <pid>        # Send SIGKILL to a process
kill -SIGINT <pid>   # Send SIGINT (Ctrl+C) to a process

# Common Signals
# SIGINT - Interrupt (Ctrl+C)
# SIGTERM - Termination request
# SIGKILL - Force kill
# SIGUSR1 - User defined signal 1
# SIGUSR2 - User defined signal 2

# 4. MESSAGE QUEUES
# Messages are sent as discrete units
# Message queues allow multiple processes to exchange data
ipcmk -Q             # Create a message queue
ipcs -q              # Show message queues
ipcrm -q <msqid>     # Remove message queue
manishaa@localhost:~$
```

```
manishaa@localhost:~$ # IPC (Inter-Process Communication) COMMENTS - PART 4

# 7. SOCKETS
# Used for IPC between processes on the same machine
# Can also be used for network communication

# Types:
# AF_UNIX - Local (IPC between processes)
# AF_INET - Network communication
# SOCK_STREAM - Connection oriented (like TCP)
# SOCK_DGRAM - Connectionless (like UDP)

# Create a UNIX domain socket (in C)
int sockfd = socket(AF_UNIX, SOCK_STREAM, 0); # Create socket
struct sockaddr_un addr;                      # Socket address
bind(sockfd, (struct sockaddr*)&addr, sizeof(addr)); # Bind socket
listen(sockfd, 5);                            # listen
accept(sockfd, NULL, NULL);                    # Accept connection

# SUMMARY
# IPC mechanisms help processes to communicate and synchronize.
# Choose the right one based on the requirement: speed, data size, direction, et.

manishaa@localhost:~$
```